

ESTRUTURA DE DADOS PILHA

Prof. Joaquim Uchôa
Profa. Juliana Gregghi
Prof. Renato Ramos



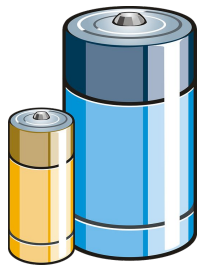
- Definição e Visão Geral
- Operações Básicas sobre Pilhas
- Informação Importante
- Implementação Utilizando Vetores

DEFINIÇÃO E VISÃO GERAL



DEFINIÇÃO

- De acordo com o dicionário o que é uma pilha?



DEFINIÇÃO

- De acordo com o dicionário o que é uma pilha?¹
 - Porção de coisas dispostas umas sobre as outras (monte);
 - Dispositivo que transforma a energia química em energia elétrica;
 - Indivíduo irritado ou nervoso
 - Caixa que contém dentro de si outras caixas de várias dimensões, acondicionadas, umas dentro das outras, de acordo com os tamanhos

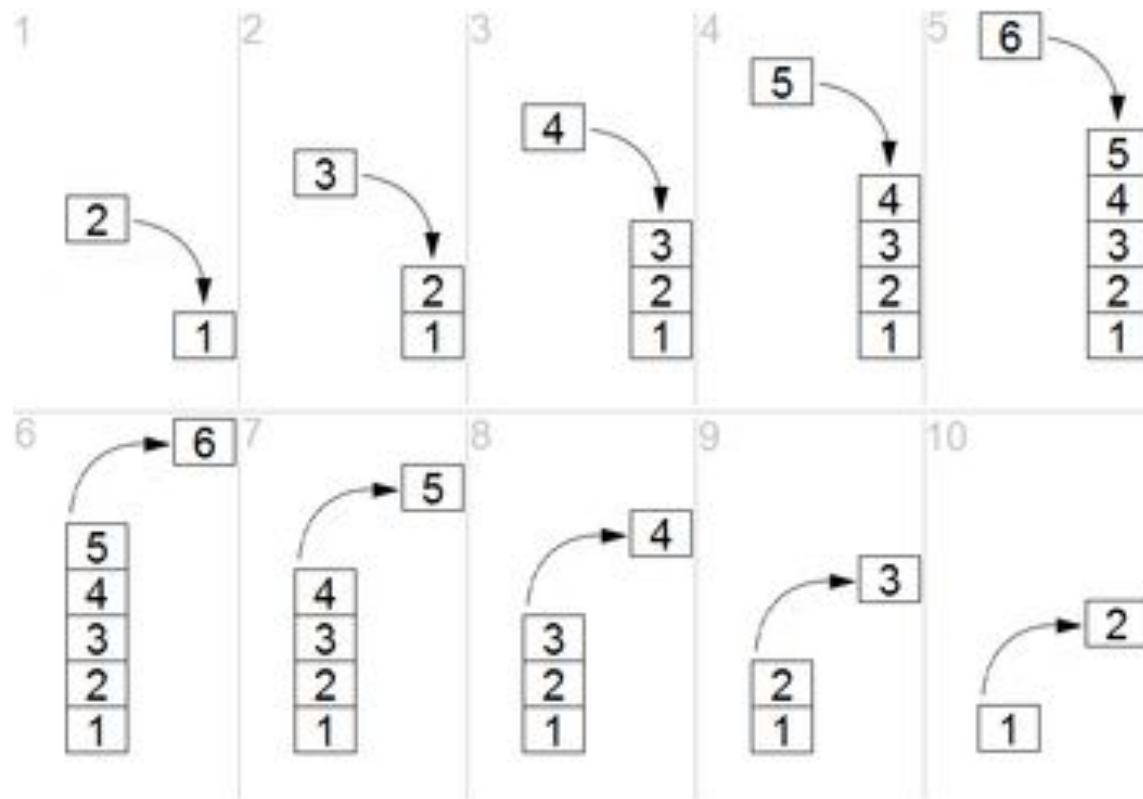
¹Segundo dicionário michaelis on-line <http://michaelis.uol.com.br/busca?id=OWQE> 5

DEFINIÇÃO

- E em Computação?
 - É uma estrutura de dados que assume a ideia de monte para armazenar as informações.
 - É baseada no princípio "*Last In First Out*" (LIFO)
 - O último inserido será o primeiro a ser retirado.
 - **A manipulação é feita somente em uma das extremidades.**
 - **Não é possível o acesso direto aos demais dados.**

DEFINIÇÃO

Ideia Básica



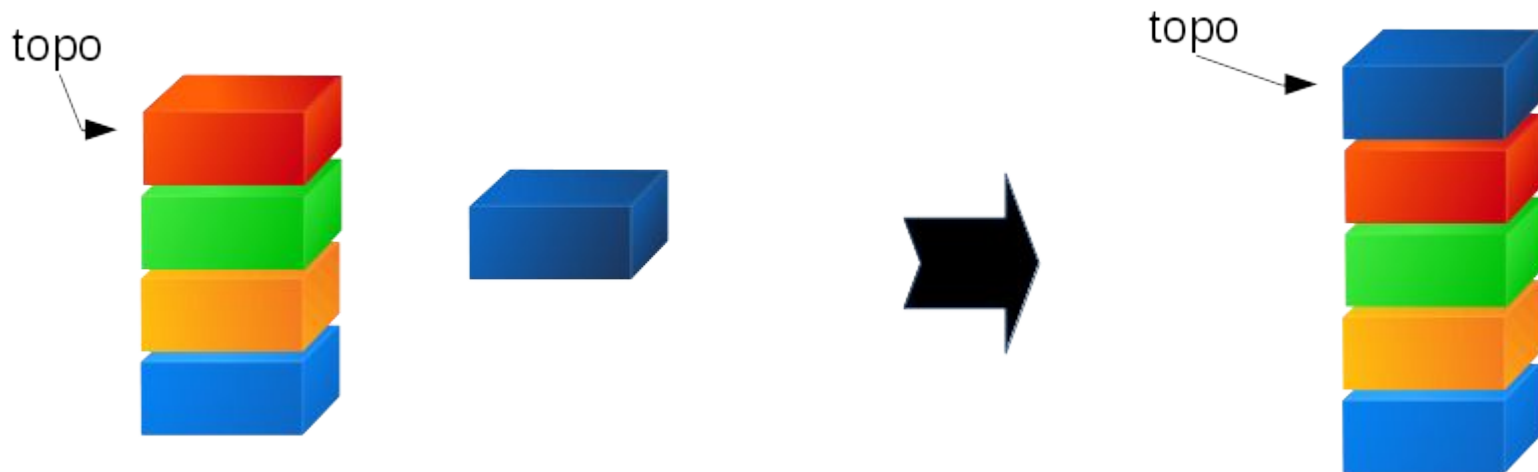
APLICAÇÕES TRADICIONAIS DE PILHAS

Pilhas são utilizadas em diversas situações na computação. Por exemplo, são utilizadas no **processo recursivo** e nas seguintes aplicações:

- Calculadora para expressões matemáticas;
- Conversão de número decimal para binário;
- Retirada de mercadorias de um caminhão de entregas;
- Mecanismo de fazer/desfazer de editores diversos;
- Mecanismo de navegação de páginas na Internet (avançar e retornar).

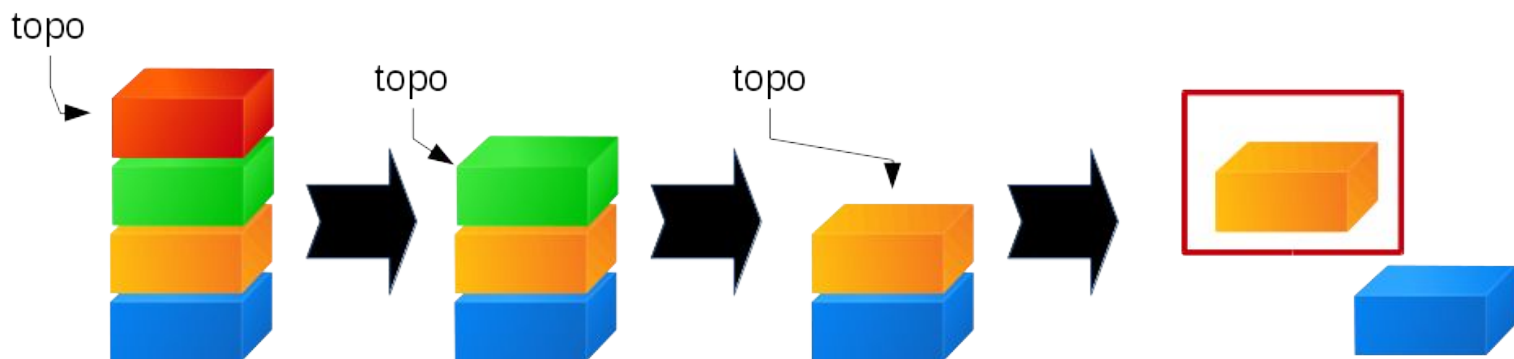
INSERÇÃO NA PILHA

A inserção deve ser sempre na posição à frente (ou acima) do último elemento inserido.



RETIRADA DE ELEMENTOS NA PILHA

A retirada de um elemento na pilha é sempre daquele que foi inserido por último. Caso se deseje acessar, por exemplo, o conteúdo de um nó intermediário, todos os demais devem ser desempilhados antes.



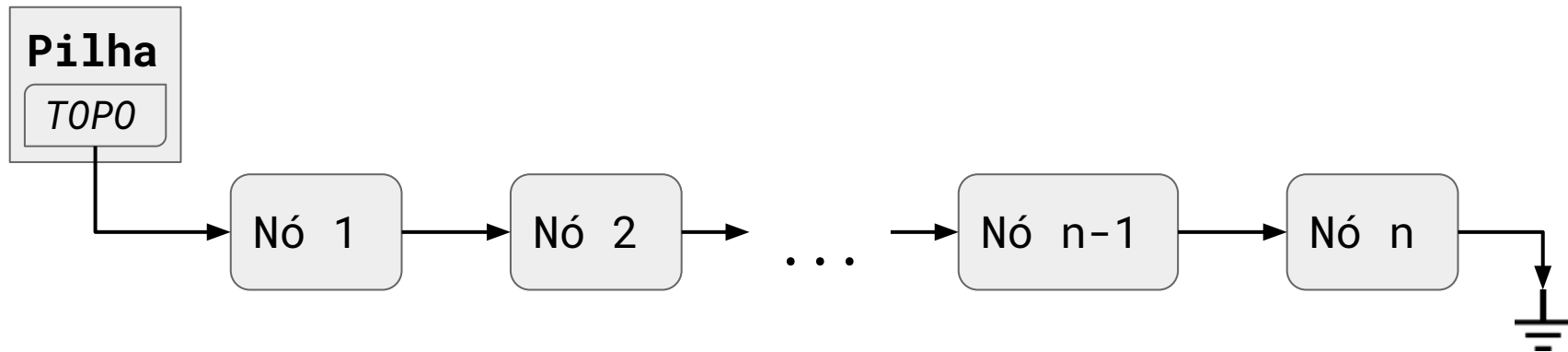
FORMAS DE IMPLEMENTAÇÃO

Uma pilha pode ser implementada de diferentes maneiras, as mais comuns são:

- Implementação usando encadeamento de nós (elementos) da pilha;
- Implementação usando arranjo para armazenamento dos elementos.

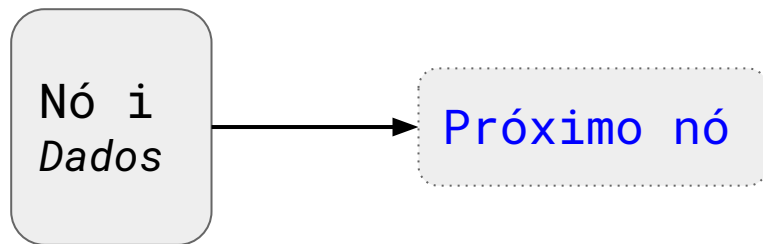
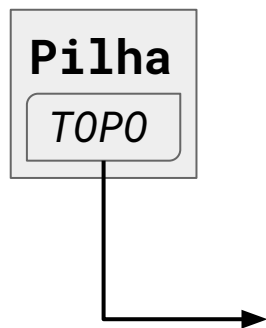
IMPLEMENTAÇÃO USANDO ENCADEAMENTO - I

A implementação mais usual de pilha é a usando encadeamento. Nessa implementação, cada nó contém informações e aponta para o próximo nó. O último nó da pilha aponta para NULO, indicando seu término. A pilha sabe apenas onde está o topo.



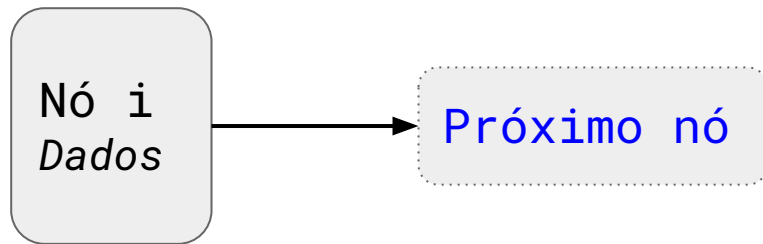
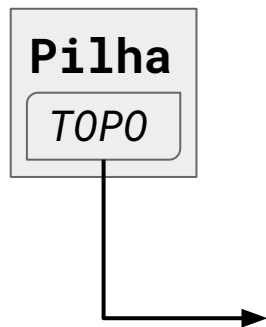
IMPLEMENTAÇÃO USANDO ENCADEAMENTO - II

Em uma implementação por encadeamento, uma pilha contém um apontador para o topo e cada nó contém o dado armazenado e um apontador para o próximo nó na estrutura.



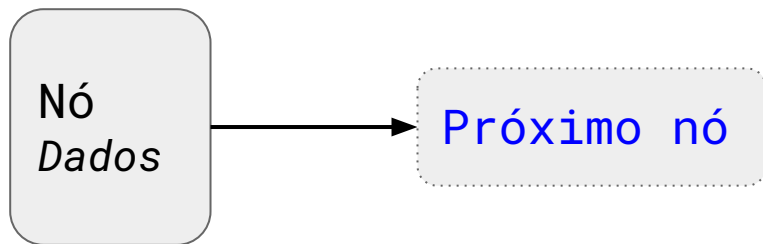
IMPLEMENTAÇÃO USANDO ENCADEAMENTO - III

Pilha e nós são implementados tradicionalmente como estruturas ou classes. Usaremos a segunda abordagem, usando uma classe para pilha e outra para os nós.



CLASSE NOH

Cada nó da pilha possui atributos com os dados armazenados, que pode ser desde um simples inteiro até uma estrutura ou outra classe. Além disso, possui um apontador para o próximo nó.

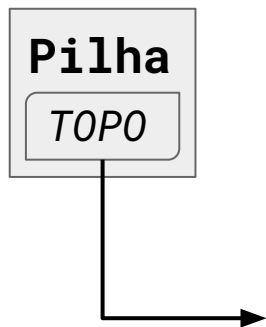


class
noh

```
int dado;  
noh* proximo;
```

CLASSE PILHA

A classe pilha contém geralmente, além do apontador para o topo, um atributo para informar a quantidade de elementos (nós) que ela possui. Outros atributos podem ser necessários, dependendo do problema resolvido.



```
class  
pilha
```

```
int tamanho;  
noh* topo;
```

OPERAÇÕES BÁSICAS SOBRE PILHAS



OPERAÇÕES BÁSICAS SOBRE PILHAS

Uma pilha possui um conjunto de operações básicas, para seu correto uso:

- Criação / destruição
- Inserção (ou empilhamento)
- Remoção (ou desempilhamento)

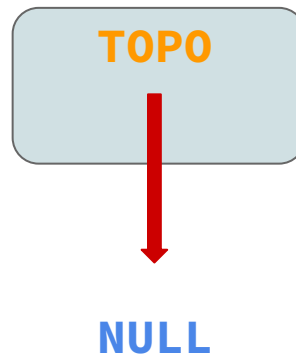
CRIAÇÃO DA PILHA

criarPilha():

```
topo ← NULL;
```

```
// informação opcional
```

```
tamanho ← 0;
```



A criação da pilha consiste basicamente em garantir que seu topo não aponte para uma região válida de memória.

INSERÇÃO DE ELEMENTO

push(valor):

```
novo ← criar_noh(valor);
```

```
novo.proximo ← topo;
```

```
topo ← novo;
```

```
tamanho++;
```

A inserção em uma pilha, consiste em criar um novo nó com o dado a ser armazenado e posteriormente colocá-lo no topo. Assim, seu próximo elemento é o topo antigo e o topo passa a ser o novo elemento inserido.

OPERAÇÕES BÁSICAS SOBRE PILHAS

Inserção ou Empilhamento (PUSH)



NULL

Situação Inicial

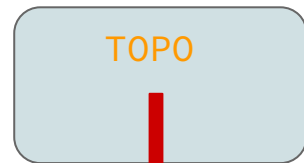


NULL



NULL

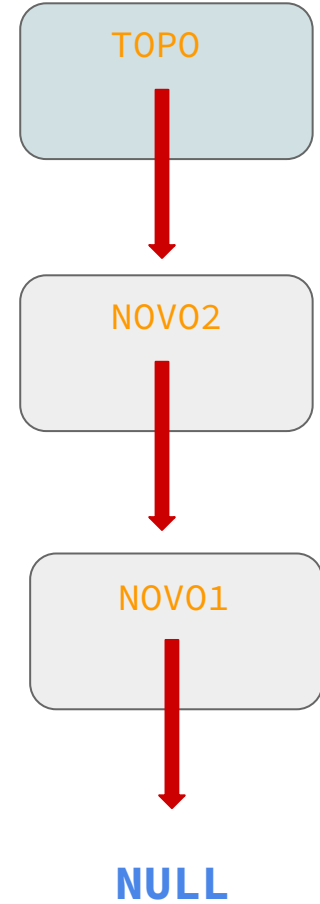
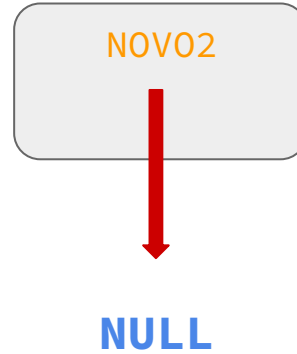
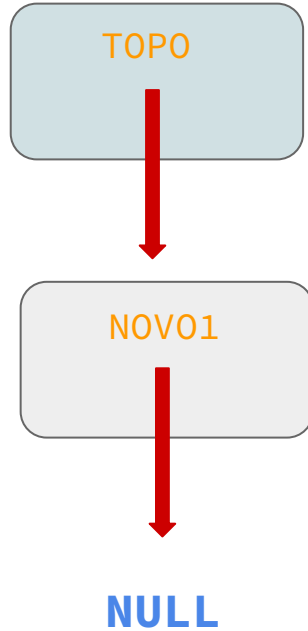
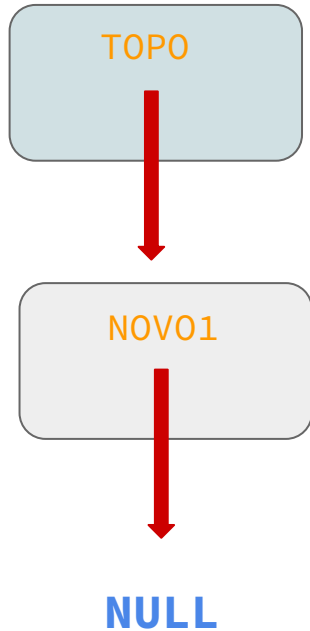
Inserindo o elemento NOVO1



NULL

OPERAÇÕES BÁSICAS SOBRE PILHAS

Inserção ou Empilhamento (PUSH)



INSERÇÃO DE ELEMENTO

push(valor):

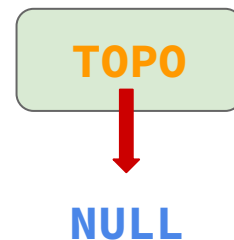
```
novo ← criar_noh(valor);
```

```
novo.proximo ← topo;
```

```
topo ← novo;
```

```
tamanho++;
```

valor=23



INSERÇÃO DE ELEMENTO

push(valor):

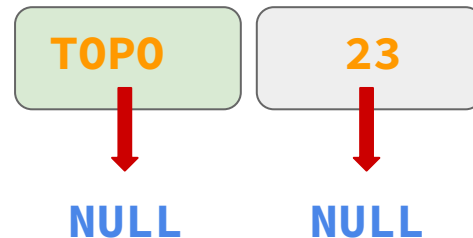
```
novo ← criar_noh(valor);
```

```
novo.proximo ← topo;
```

```
topo ← novo;
```

```
tamanho++;
```

valor=23



INSERÇÃO DE ELEMENTO

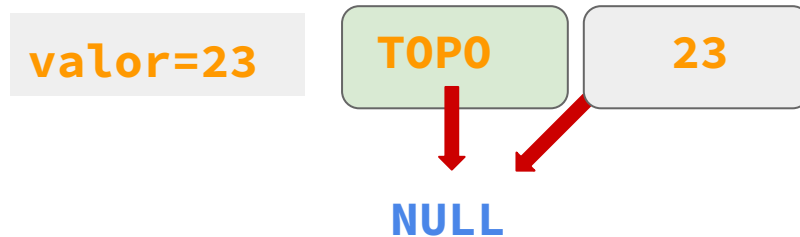
push(valor):

novo ← criar_noh(valor);

novo.proximo ← topo;

topo ← novo;

tamanho++;



INSERÇÃO DE ELEMENTO

push(valor):

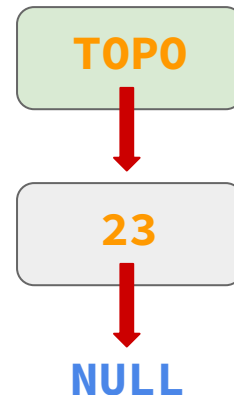
novo ← criar_noh(valor);

novo.proximo ← topo;

topo ← novo;

tamanho++;

valor=23



INSERÇÃO DE ELEMENTO

push(valor):

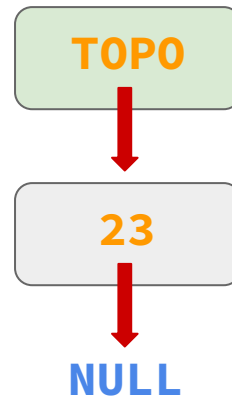
novo ← criar_noh(valor);

novo.proximo ← topo;

topo ← novo;

tamanho++;

valor=23



tamanho=1

INSERÇÃO DE ELEMENTO

push(valor):

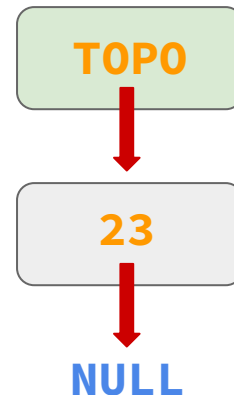
```
novo ← criar_noh(valor);
```

```
novo.proximo ← topo;
```

```
topo ← novo;
```

```
tamanho++;
```

valor=45



INSERÇÃO DE ELEMENTO

push(valor):

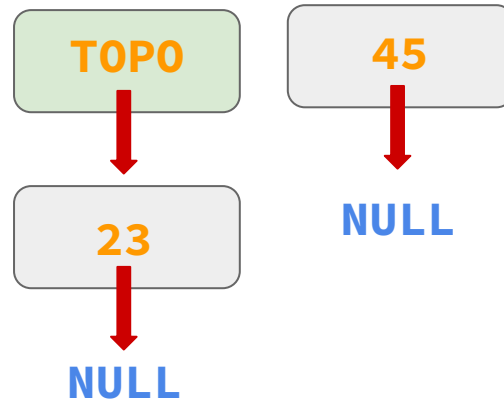
novo ← criar_noh(valor);

novo.proximo ← topo;

topo ← novo;

tamanho++;

valor=45



INSERÇÃO DE ELEMENTO

push(valor):

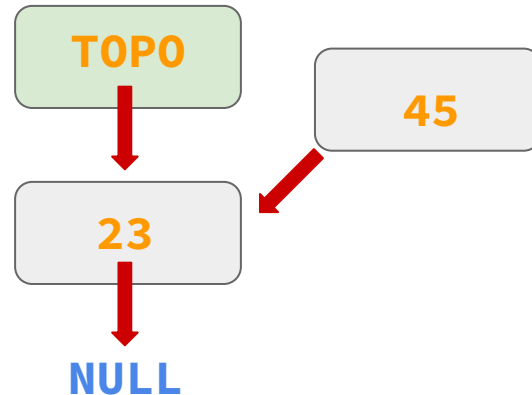
novo ← criar_noh(valor);

novo.proximo ← topo;

topo ← novo;

tamanho++;

valor=45



INSERÇÃO DE ELEMENTO

push(valor):

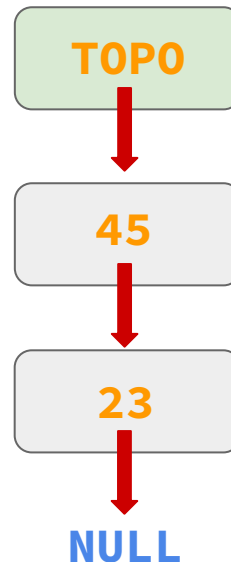
novo ← criar_noh(valor);

novo.proximo ← topo;

topo ← novo;

tamanho++;

valor=45



INSERÇÃO DE ELEMENTO

push(valor):

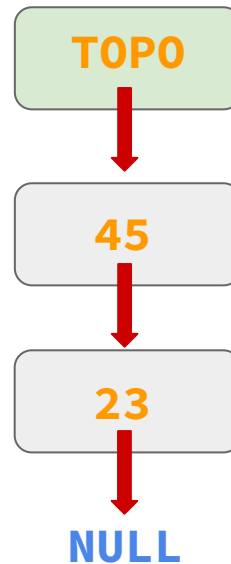
novo ← criar_noh(valor);

novo.proximo ← topo;

topo ← novo;

tamanho++;

valor=45



tamanho=2

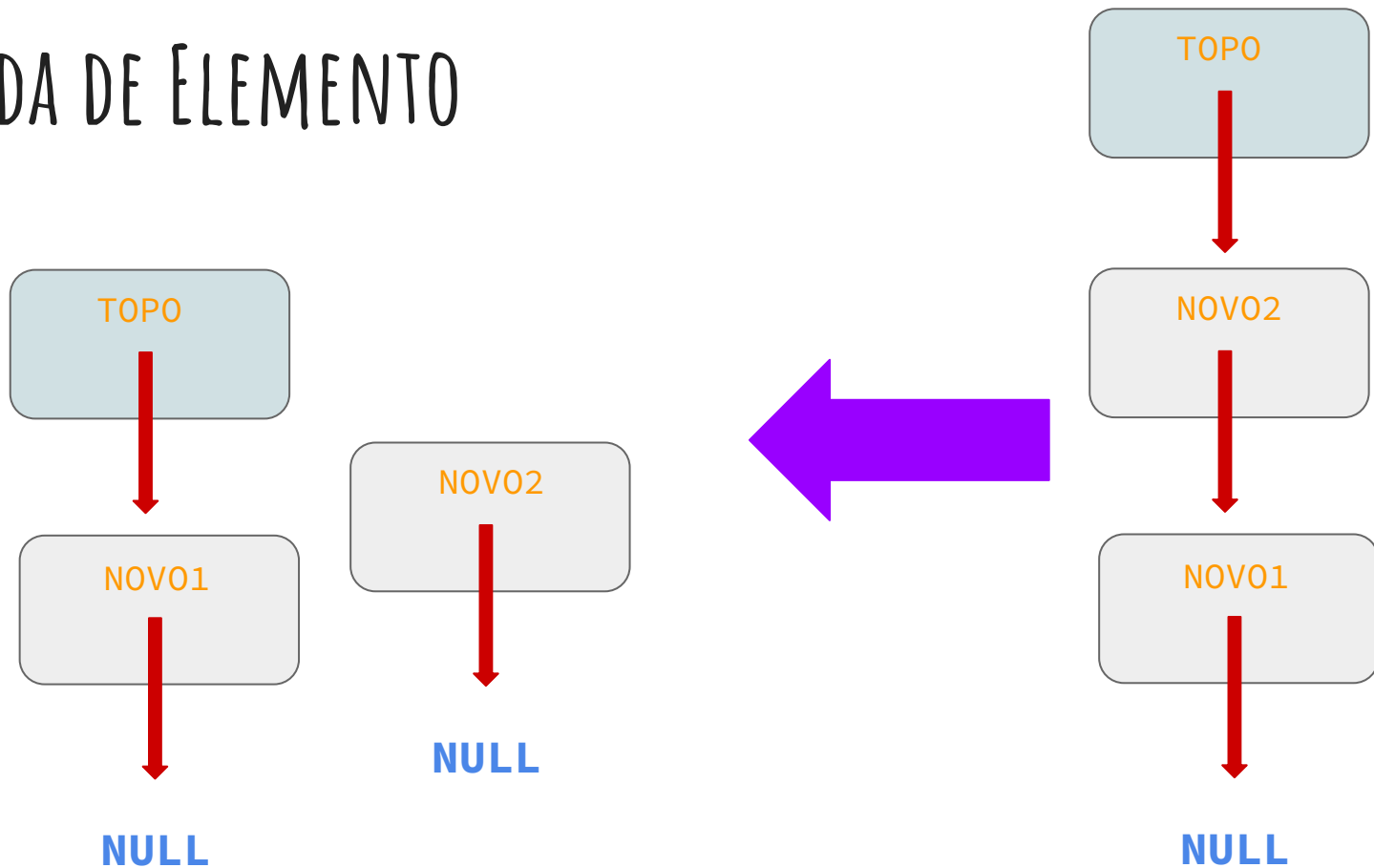
RETIRADA DE ELEMENTO

pop():

```
aux ← topo;  
dado ← aux.valor;  
topo ← aux.proximo;  
apagar(aux);  
tamanho--;  
// faz ação desejada  
// (e.g.: retorno)  
efetuaAcao(dado);
```

A retirada de um elemento da pilha consiste em acessar o seu topo, retornando-o para a ação desejada. Nesse caso, o novo topo passa a ser o próximo nó do topo anterior, que será removido do sistema.

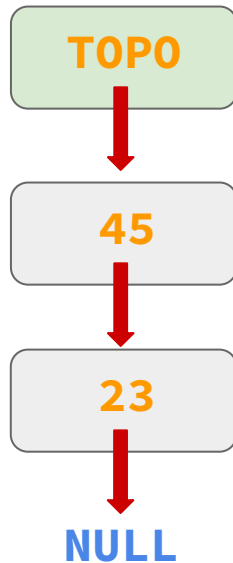
RETIRADA DE ELEMENTO



RETIRADA DE ELEMENTO

pop():

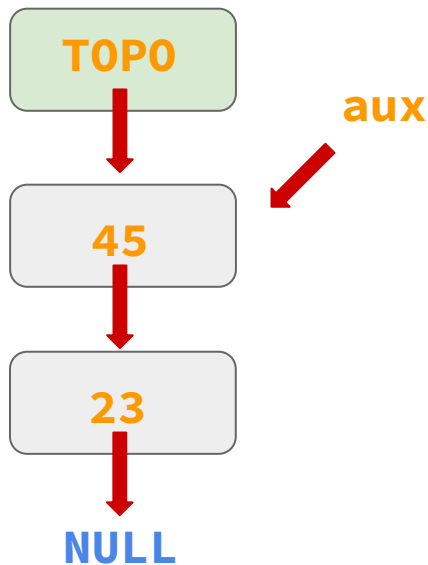
```
aux ← topo;  
dado ← aux.valor;  
topo ← aux.proximo;  
apagar(aux);  
tamanho--;  
// faz ação desejada  
// (e.g.: retorno)  
efetuaAcao(dado);
```



RETIRADA DE ELEMENTO

pop():

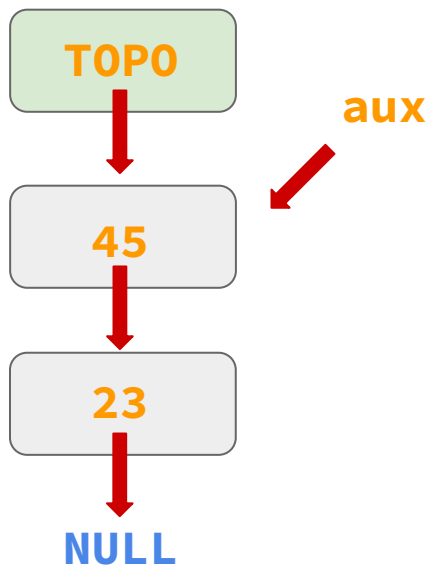
```
aux ← topo;  
dado ← aux.valor;  
topo ← aux.proximo;  
apagar(aux);  
tamanho--;  
// faz ação desejada  
// (e.g.: retorno)  
efetuaAcao(dado);
```



RETIRADA DE ELEMENTO

pop():

```
aux ← topo;  
dado ← aux.valor;  
topo ← aux.proximo;  
apagar(aux);  
tamanho--;  
// faz ação desejada  
// (e.g.: retorno)  
efetuaAcao(dado);
```

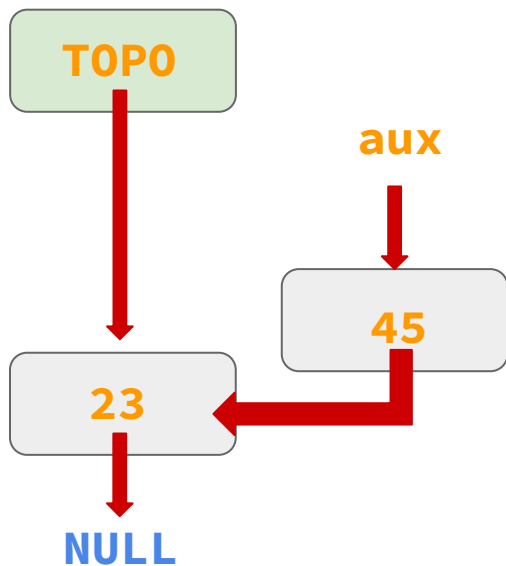


dado=45

RETIRADA DE ELEMENTO

pop():

```
aux ← topo;  
dado ← aux.valor;  
topo ← aux.proximo;  
apagar(aux);  
tamanho--;  
// faz ação desejada  
// (e.g.: retorno)  
efetuaAcao(dado);
```

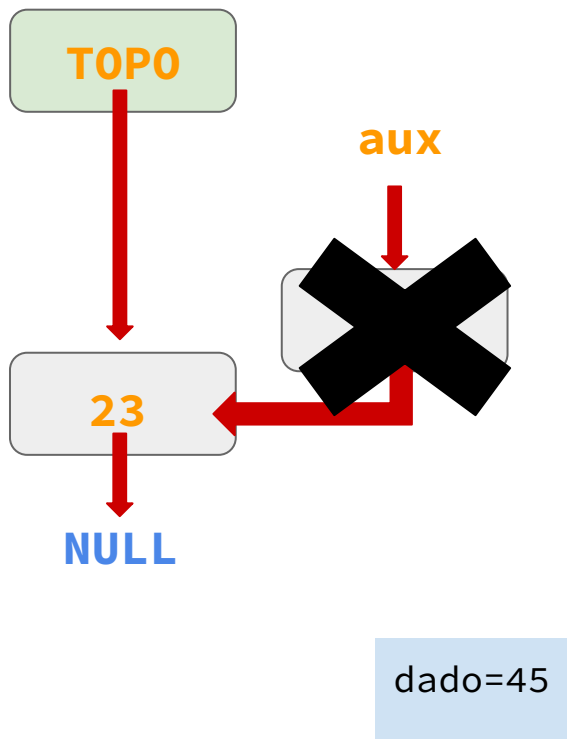


dado=45

RETIRADA DE ELEMENTO

pop():

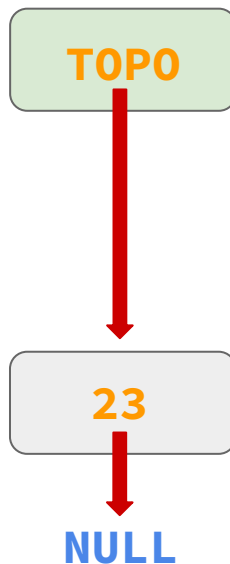
```
aux ← topo;  
dado ← aux.valor;  
topo ← aux.proximo;  
apagar(aux);  
tamanho--;  
// faz ação desejada  
// (e.g.: retorno)  
efetuaAcao(dado);
```



RETIRADA DE ELEMENTO

pop():

```
aux ← topo;  
dado ← aux.valor;  
topo ← aux.proximo;  
apagar(aux);  
tamanho--;  
// faz ação desejada  
// (e.g.: retorno)  
efetuaAcao(dado);
```

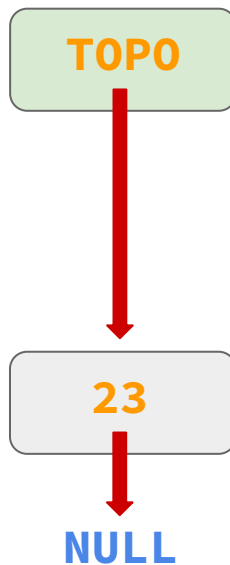


dado=45
tamanho=1

RETIRADA DE ELEMENTO

pop():

```
aux ← topo;  
dado ← aux.valor;  
topo ← aux.proximo;  
apagar(aux);  
tamanho--;  
// faz ação desejada  
// (e.g.: retorno)  
efetuaAcao(dado);
```



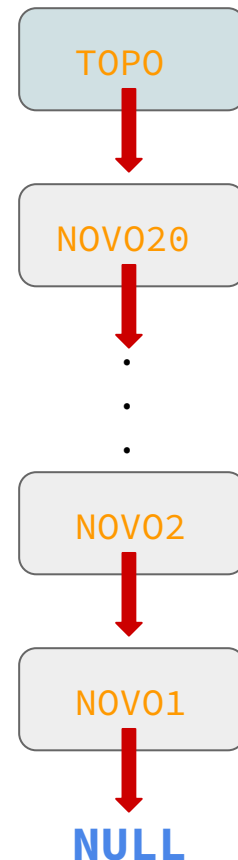
dado=45
tamanho=1

INFORMAÇÃO
IMPORTANTE



INFORMAÇÃO IMPORTANTE

- O acesso à Pilha deve ser sempre feito pelo TOPO
- As ligações entre os nós é para que a estrutura permaneça integrada.



INFORMAÇÃO IMPORTANTE

Mas professor, dessa forma eu só posso acessar o elemento que o TOP0 aponta? E se, por exemplo, na estrutura anterior, eu quiseser utilizar o elemento NOV012, o que fazer?



INFORMAÇÃO IMPORTANTE

Pela definição, não é possível acessar os outros elementos que não estão no topo. Assim, não é possível desenvolver uma função da pilha que resolva isso.

Mas você pode retirar todos os elementos do topo até encontrar o elemento desejado. Vamos deixar claro, que essa situação deve ser feita via retirada de elementos, **não** pode ser um procedimento da pilha.

OUTRAS OPERAÇÕES EM PILHAS - I

Além das operações básicas apresentadas, as pilhas podem implementar:

- verificar o número de elementos da pilha;
- verificar se a pilha está ou não vazia;
- retirar todos os elementos da pilha;
- opcionalmente acessar o topo da pilha;
- opcionalmente depurar/imprimir a pilha.

OUTRAS OPERAÇÕES EM PILHAS - II

- A retirada de todos os elementos da pilha é por meio de desempilhamento, um por um.
- O destrutor é implementado geralmente usando o método para retirar todos os elementos da pilha.

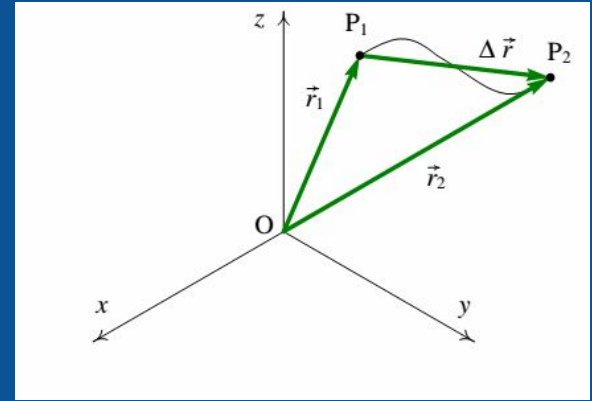
OUTRAS OPERAÇÕES EM PILHAS - III

- O acesso ao topo (espia) costuma ser disponibilizado em várias implementações. Nesse caso, há o acesso ao dado, mas sem sua retirada, que continua no topo da pilha.

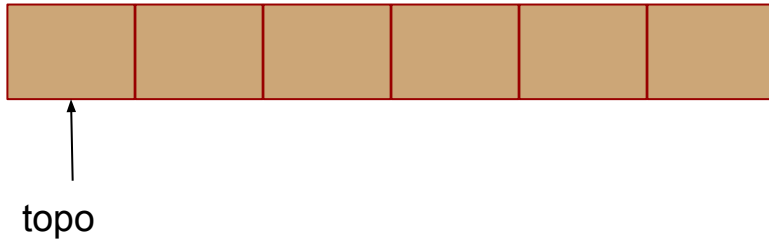
OUTRAS OPERAÇÕES EM PILHAS - IV

- Implementações iniciais às vezes fazem uso de mecanismos de percorrimto para impressão/depuração dos dados. Esse método é considerado uma **quebra de estrutura**, pois acessa os elementos intermediários sem desempilhamento. Assim, esse recurso só pode ser usado, para fins didáticos ou de depuração.

IMPLEMENTAÇÃO UTILIZANDO VETOR



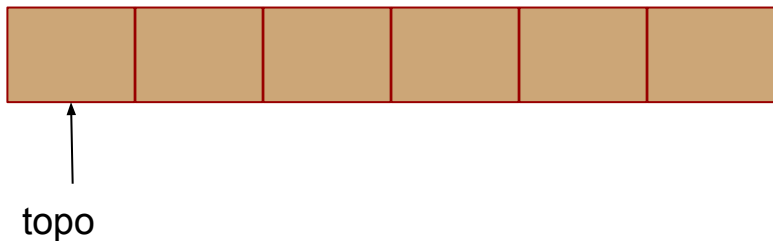
IMPLEMENTAÇÃO UTILIZANDO VETOR



Vantagem

- Operações rápidas (acesso, empilhamento, desempilhamento).

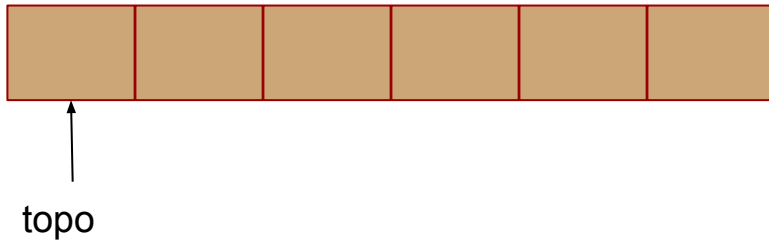
IMPLEMENTAÇÃO UTILIZANDO VETOR



Desvantagem

- Espaço alocado sem necessidade;
- Impossibilidade de aumentar a capacidade da pilha sem redimensionamento do vetor.

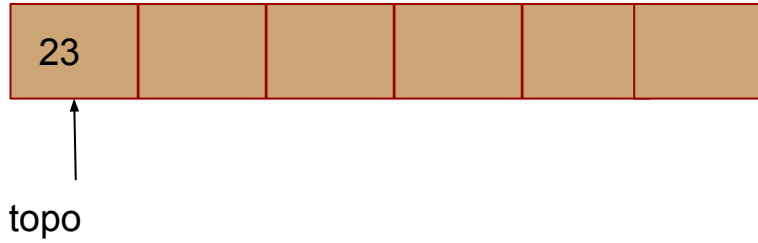
IMPLEMENTAÇÃO UTILIZANDO VETOR



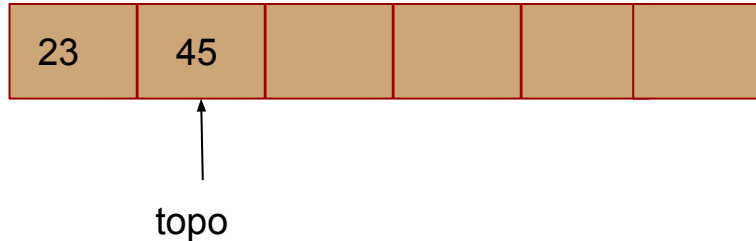
A implementação em vetor utiliza um atributo para armazenar o índice do vetor que contém o topo da pilha. Em uma inserção, o topo avança uma posição. Em uma retirada, o topo retorna uma posição.

CONSIDERE OS ELEMENTOS
23, 45, 11 E 89.

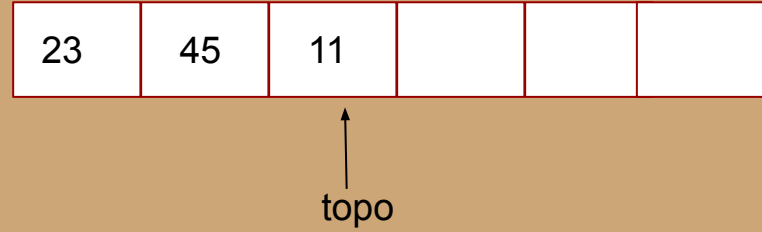
Inserir elemento na pilha



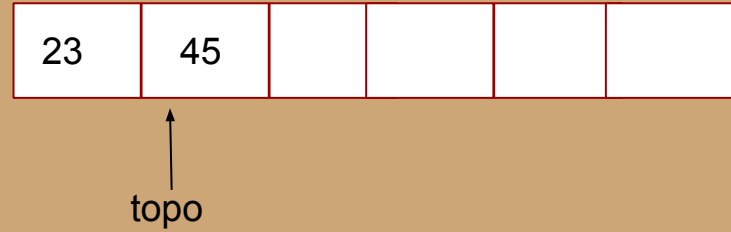
Inserir elemento na pilha



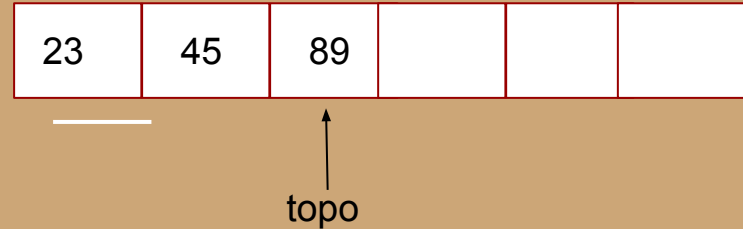
Inserir elemento na pilha



Remover elemento da pilha



Inserir elemento na pilha



UM EXEMPLO:
NOTAÇÃO PÓSFIXA



EXPRESSÃO EM NOTAÇÃO PÓSFIXA - I

Um dos problemas ao tratar expressões matemáticas computacionalmente é a precedência de operadores:

$2+4*2 = 10?$ Ou $2+4*2 = 12?$

Por conta disso, usamos geralmente parênteses:

$2+(4*2) = 10$ e $(2+4)*2 = 12$

Mas internamente, geralmente se usam outras formas de representação.

EXPRESSÃO EM NOTAÇÃO PÓSFIXA - II

Uma das formas de representar expressões sem o problema com precedência de operadores é o uso de notação pósfixa. Nesse caso, o operador é inserido após os operandos a que deve ser aplicado. Por exemplo:

$2+(4*2)$ seria escrito como 2 4 2 * +

$(2+4)*2$ seria escrito como 2 4 + 2 *

EXPRESSÃO EM NOTAÇÃO PÓSFIXA - III

A ideia é que o processamento é feito da seguinte forma: sempre que um operador é lido, ele é aplicado aos dois operandos anteriores. Então vejamos:

2 4 2 * + => ao percorrer, encontramos o *, então ele é aplicado a 4 e 2, produzindo 8. Na sequência, o + é aplicado ao 8 e o 2 inicial, produzindo 10.

EXPRESSÃO EM NOTAÇÃO PÓSFIXA - IV

2 4 + 2 * => ao percorrer, encontramos o +, então ele é aplicado a 2 e 4, produzindo 6. Na sequência, o * é aplicado ao 6 e o 2 final, produzindo 12.

USANDO PILHAS PARA EXPRESSÃO EM NOTACÃO PÓSFIXA

Uma das vantagens dessa notação é a possibilidade de uso da estrutura de dados pilha para resolver a expressão.

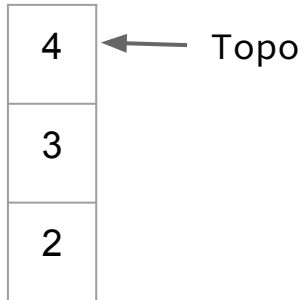
Para isso, basta ir empilhando os valores e, ao encontrar um operador, fazer o desempilhamento dos valores associados.

EXEMPLO - I

Considere a seguinte expressão pós-fixada:

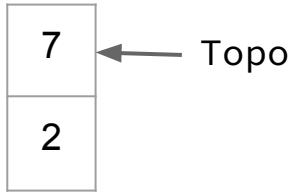
2 3 4 + *

Para calcular o resultado utilizando pilha, empilhamos inicialmente os valores 2, 3 e 4:



EXEMPLO - II

Ao encontrar o +, o 4 e o 3 são desempilhados e a operação de soma é realizada. O resultado é novamente empilhado:



Por fim, ao encontrar o *, o 7 e o 2 são desempilhados, a operação é realizada e tem-se o resultado final 14, que será o último elemento na pilha.

SOBRE O MATERIAL



SOBRE ESTE MATERIAL

Material produzido coletivamente, principalmente pelos seguintes professores do DAC/UFLA:

- Joaquim Quinteiro Uchôa
- Juliana Galvani Greggi
- Renato Ramos da Silva

Inclui contribuições de outros professores do setor de Fundamentos de Programação do DAC/UFLA.

Esta obra está licenciado com uma Licença [Creative Commons Atribuição 4.0 Internacional](https://creativecommons.org/licenses/by/4.0/).